PROJECT: SHOPEASE

Microservices-Based E-Commerce Management System

Day 1 – Study of Microservices Architecture: Learning, Documentation and Architecture Design

Field	Details
Submitted by	Baskar P
Date	29-10-2026
Task	Day 1 – Study Microservices Architecture
Technology stack	Python, FastAPI, MySQL, SQLAlchemy ORM, Alembic, Pydantic, JWT, Docker, Docker Compose, Git and GitHub
Deliverables covered	Comparison document, service responsibilities, database ownership diagram, REST API notes, service-to-service workflow, complete architecture diagram

1. Introduction

ShopEase is an enterprise-style e-commerce backend built with a microservices architecture. Instead of one large application, the system is divided into small services. Each service has a single business responsibility, its own database, and communicates with the other services through REST APIs.

Day 1 is dedicated to learning, research, documentation and architecture design. No application code is written at this stage. The purpose of this document is to show a clear understanding of the following topics:

- Monolithic and microservices architecture and the differences between them
- The responsibilities of the six components: Authentication, User, Product, Order, Payment and the API Gateway
- Database ownership, isolation and service independence
- REST APIs, HTTP methods, JSON bodies and status codes
- Service-to-service communication and error handling
- The complete ShopEase system architecture

2. Monolithic vs. Microservices Architecture

2.1 Monolithic Architecture

A monolithic application is built as a single codebase and deployed as one unit. All modules (users, products, orders, payments) run in the same process and normally share one database. It is simple to build, test and deploy at the beginning, but becomes difficult to scale and maintain as the application and the team grow.

2.2 Microservices Architecture

A microservices application is a collection of small, independent services. Each service focuses on one business capability, runs in its own process or container, owns its data, and is deployed independently. Services talk to each other over the network, usually through REST APIs.

2.3 Comparison

Aspect	Monolithic	Microservices
Structure	Single codebase and one deployable unit	Many small services, each with its own codebase
Database	One shared database for all modules	Database per service
Scalability	Whole application must be scaled together, even if only one module is busy	Only the busy service is scaled (for example the Product Service during a sale)
Deployment	Any change needs the entire application to be rebuilt and redeployed	Each service is built, tested and deployed independently
Fault isolation	A bug or crash in one module can bring down the whole application	A failure stays inside one service; others continue to work
Technology	Single technology stack for everything	Each service can use the stack that suits it best
Development speed	Fast at the start; slows down as the codebase grows	Slower at the start; faster at scale with parallel teams
Team structure	Large team working on one codebase, frequent merge conflicts	Small teams own individual services end to end
Testing	Simple end-to-end testing in one place	Each service is tested alone; integration testing is more complex
Operational complexity	Low – one application to run and monitor	High – many services, networking, monitoring, data consistency
Communication	In-process function calls (fast)	Network calls over REST (slower, can fail)

2.4 Advantages and Disadvantages

	Advantages	Disadvantages
Monolithic	Simple to develop and deploy Easy debugging and testing in one place No network latency between modules	Difficult to scale a single module One failure can affect everything Hard to adopt new technologies Long build and deployment times as it grows
Microservices	Independent scaling and deployment Fault isolation Technology flexibility Faster releases and team autonomy	More complex to design and operate Network failures and latency Data consistency across services is harder Needs monitoring, Docker and DevOps practices

2.5 Deployment Approaches

- **Monolithic:** build one package and deploy it on a server. Scaling means running more copies of the entire application behind a load balancer.
- **Microservices:** each service is packaged as its own Docker image. Docker Compose runs all containers together during development, and each container can be scaled or redeployed on its own.

2.6 Why Microservices Suit Large Enterprise Applications

- **Independent scaling:** heavily used services such as Product or Order can be scaled without scaling the rest.
- **Fault isolation:** if the Payment Service is down, customers can still browse products.
- **Faster delivery:** small services can be updated and released without touching the whole system.
- **Team autonomy:** different teams own different services and work in parallel.
- **Technology flexibility:** each service can evolve and adopt new tools independently.
- **Maintainability:** small codebases are easier to understand, test and replace.

3. Service Responsibilities

ShopEase contains five business microservices and one API Gateway. Every service has a clearly defined responsibility, which keeps the system easy to understand and maintain.

Service	Responsibilities	Owns data	Calls / used by
Authentication Service	User registration Login and password hashing JWT access token generation and validation Role information in the token	auth_db (credentials, roles)	Used by the API Gateway to verify tokens

Service	Responsibilities	Owns data	Calls / used by
User Service	User profile management Address management (add, update, delete) Provide user details to other services	user_db (users, addresses)	Called by the Order Service when delivery address is needed
Product Service	Product catalog Categories Inventory and stock levels Stock availability validation	product_db (products, categories, inventory)	Called by the Order Service
Order Service	Create orders Order history Order status management Coordinate with Product and Payment services	order_db (orders, order_items)	Calls Product and Payment services
Payment Service	Payment records Transaction history Payment status (SUCCESS, FAILED)	payment_db (payments, transactions)	Called by the Order Service
API Gateway	Single entry point for clients Request routing to services JWT verification before forwarding Central place for logging and rate limiting	No database	Sits between the client and all services

3.1 Responsibility Details

Authentication Service. Handles identity only. A new user registers with an email and password; the password is stored as a hash, never as plain text. On login the service issues a signed JWT that contains the user id and role. Every other request uses this token to prove who the user is.

User Service. Stores everything about a customer that is not a credential: name, phone, and multiple delivery addresses. Keeping this separate from authentication means login logic and profile logic can change independently.

Product Service. Owns the catalog and the stock count. It is the only service allowed to change inventory, which keeps stock data consistent.

Order Service. The coordinator of the purchase flow. It validates products, saves the order, requests payment, and updates the order status based on the result.

Payment Service. Records each payment attempt and its result. It maintains transaction history so that every order can be traced to a payment record.

API Gateway. Clients never call internal services directly. They send every request to the gateway, which checks the token and routes the request to the correct service. This hides the internal structure and gives one secure entry point.

4. Database Ownership

In a microservices system every service owns its own database. No other service is allowed to read from or write to it. The API Gateway does not need a database because it stores no business data.

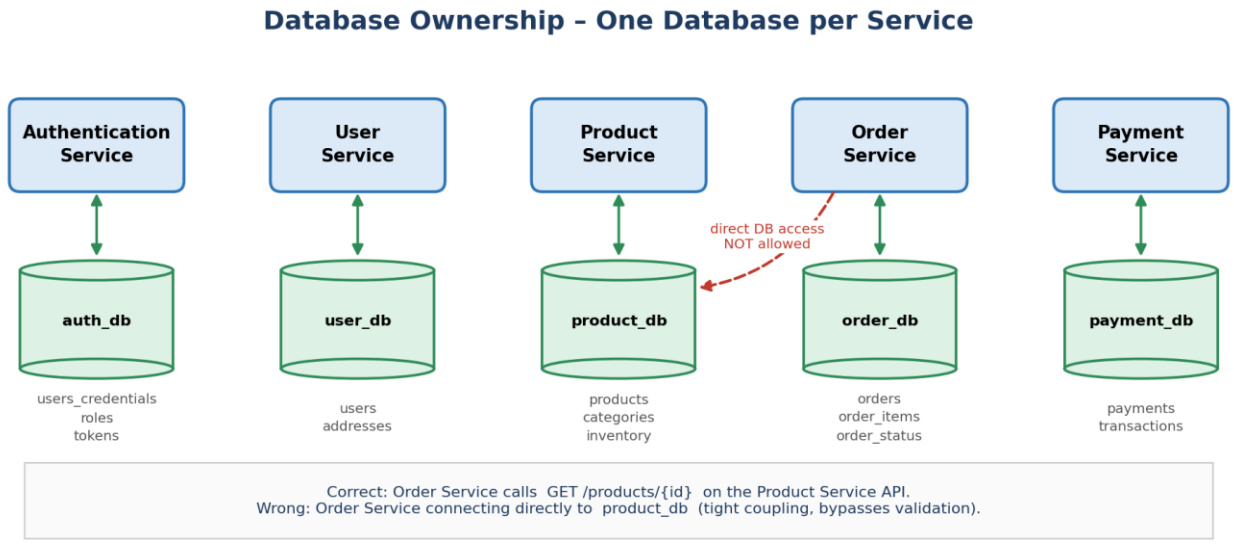


Figure 1: Database ownership – each service has its own database

4.1 Database Isolation

Each service has its own schema and its own database. A change to the Product tables, such as renaming a column, cannot break the Order Service because the Order Service never touches those tables. Problems like locks, slow queries or corrupted data stay inside one service.

4.2 Service Independence

- Each service can be developed, deployed and scaled on its own.
- Each service can choose the database design that fits its data best.
- A service can be rewritten or replaced without affecting the others as long as its API stays the same.

4.3 Why Services Use APIs Instead of Direct Database Access

- **Loose coupling:** with direct access, any schema change in one service becomes a breaking change for every service that reads it.
- **Business rules are protected:** the owning service validates data and enforces rules. Direct database access bypasses this validation.
- **Security:** a service exposes only the data it chooses to expose, instead of whole tables.
- **Independent evolution:** the owner can change its database internally while the API contract stays the same.

Example: to check stock, the Order Service calls GET /products/{id} on the Product Service. It never runs a query on product_db.

4.4 Database Summary

Service	Database	Main tables
Authentication Service	auth_db	credentials, roles, refresh tokens (if used)
User Service	user_db	users, addresses
Product Service	product_db	products, categories, inventory
Order Service	order_db	orders, order_items, order status history
Payment Service	payment_db	payments, transactions
API Gateway	None	Stateless – routing only

5. REST API and HTTP Methods

REST (Representational State Transfer) is a style of building APIs where resources are identified by URLs and manipulated using standard HTTP methods. Data is exchanged as JSON. REST is stateless: every request carries all the information needed to process it, including the JWT token.

5.1 HTTP Methods

Method	Purpose	ShopEase example
GET	Read a resource. Safe, does not change data.	GET /products – list all products
POST	Create a new resource.	POST /orders – place a new order
PUT	Replace a whole resource.	PUT /users/1 – update the entire profile
PATCH	Update part of a resource.	PATCH /products/5/stock – change only the stock
DELETE	Remove a resource.	DELETE /users/1/addresses/3 – delete an address

5.2 Request and Response Bodies (JSON)

Register a user (Authentication Service)

```
POST /auth/register
{
  "name": "Baskar P",
  "email": "baskar@example.com",
  "password": "Secret@123"
}

201 Created
{
  "id": 1,
  "email": "baskar@example.com",
  "message": "User registered successfully"
}
```

Login (Authentication Service)

```
POST /auth/login
{
  "email": "baskar@example.com",
  "password": "Secret@123"
}

200 OK
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer"
}
```

Create an order (Order Service)

```
POST /orders
Authorization: Bearer <JWT>
{
  "user_id": 1,
  "items": [
    { "product_id": 5, "quantity": 2 }
  ]
}

201 Created
{
  "order_id": 101,
  "status": "PENDING",
  "total_amount": 1998.00
}
```

Error response (product out of stock)

```
409 Conflict
{
  "detail": "Product 5 has insufficient stock"
}
```

5.3 HTTP Status Codes

Code	Meaning	When it is used in ShopEase
200 OK	Request succeeded	Successful GET, PUT or PATCH
201 Created	Resource created	New user, order or payment created
204 No Content	Success, no body returned	Successful DELETE
400 Bad Request	Invalid request	Malformed JSON or missing data
401 Unauthorized	Missing or invalid token	No JWT or expired JWT
403 Forbidden	Authenticated but not allowed	Customer trying to use an admin endpoint
404 Not Found	Resource does not exist	Order or product id not found
409 Conflict	Conflicts with current state	Duplicate email, insufficient stock
422 Unprocessable Entity	Validation error	Pydantic validation failure in FastAPI
500 Internal Server Error	Unexpected server error	Unhandled exception in a service
503 Service Unavailable	Service or dependency is down	Order Service cannot reach Payment Service

5.4 Example Endpoints for Each Microservice

Service	Method and endpoint	Description
Authentication	POST /auth/register POST /auth/login GET /auth/verify	Register a new user Login and receive a JWT Validate a token
User	GET /users/{id} PUT /users/{id} POST /users/{id}/addresses GET /users/{id}/addresses DELETE /users/{id}/addresses/{address_id}	Get profile Update profile Add an address List addresses Delete an address
Product	GET /products GET /products/{id} POST /products PUT /products/{id} PATCH /products/{id}/stock DELETE /products/{id} GET /categories	List products Get one product Add a product (admin) Update a product Update stock Delete a product List categories
Order	POST /orders GET /orders/{id} GET /orders/user/{user_id} PATCH /orders/{id}/status	Create an order Get order details Order history of a user Update order status
Payment	POST /payments GET /payments/{id} GET /payments/order/{order_id} GET /payments/user/{user_id}	Process a payment Get payment details Payment for an order Transaction history
API Gateway	/api/auth/* /api/users/* /api/products/* /api/orders/* /api/payments/*	Routes to Authentication Service Routes to User Service Routes to Product Service Routes to Order Service Routes to Payment Service

6. Service-to-Service Communication

Services communicate through synchronous REST calls over HTTP. The most important example is order creation, where the Order Service talks to the Product Service and the Payment Service.

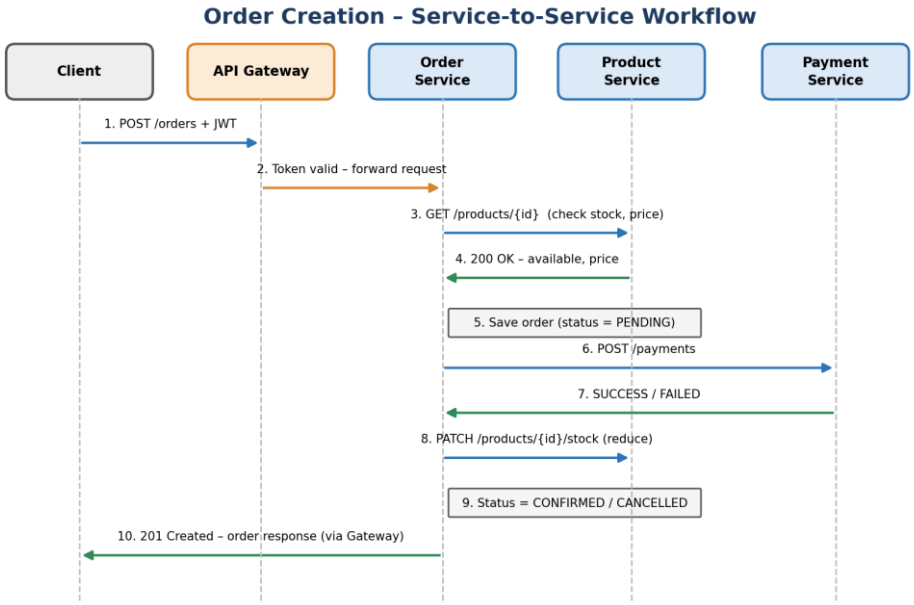


Figure 2: Order creation workflow across services

6.1 Order Creation Workflow

1. The client sends POST /orders with the JWT token to the API Gateway.
2. The Gateway validates the token (with the Authentication Service) and forwards the request to the Order Service.
3. The Order Service calls the Product Service (GET /products/{id}) to check availability and price.
4. If stock is available, the Order Service saves the order with status PENDING. If not, it returns 409 Conflict.
5. The Order Service calls the Payment Service (POST /payments) to process the payment.
6. The Payment Service records the transaction and returns SUCCESS or FAILED.
7. On SUCCESS, the Order Service reduces stock through the Product Service (PATCH /products/{id}/stock) and sets the order status to CONFIRMED. On FAILED, the status becomes CANCELLED.
8. The response is returned to the client through the API Gateway.

6.2 Order Status Lifecycle

Status	Meaning
PENDING	Order saved, waiting for payment
CONFIRMED	Payment succeeded and stock reduced
CANCELLED	Payment failed or order cancelled

Status	Meaning
FAILED	Order could not be completed because a dependent service was unavailable

6.3 Error Handling When a Dependent Service Is Unavailable

Because services communicate over a network, any call can fail. The Order Service must handle these failures safely and never lose or corrupt an order.

Scenario	Handling
Product Service is down	The order is not created. The Order Service returns 503 Service Unavailable with a clear message, and the client can retry.
Payment Service is down	The order stays in PENDING (or is marked FAILED). The payment can be retried later. Stock is not reduced until payment succeeds.
Slow response from a service	Every call has a timeout so the Order Service does not wait forever.
Temporary network error	Retry the call a few times with a short, increasing delay (exponential backoff).
Repeated failures	Circuit breaker: stop calling the failing service for a short period and fail fast instead of overloading it.
Payment succeeded but stock update failed	Compensating action: refund or cancel the payment and mark the order CANCELLED (saga pattern concept).
Duplicate payment request on retry	Use an idempotency key (for example the order id) so the same payment is never processed twice.

General practices: timeouts on every call, limited retries, circuit breaker, meaningful error messages and status codes, logging with a correlation id, and a /health endpoint on each service so failures can be detected quickly.

7. ShopEase Microservices Architecture Diagram

The diagram below shows the complete system: the client, the API Gateway, the five microservices, and the database owned by each service, along with the communication between services.

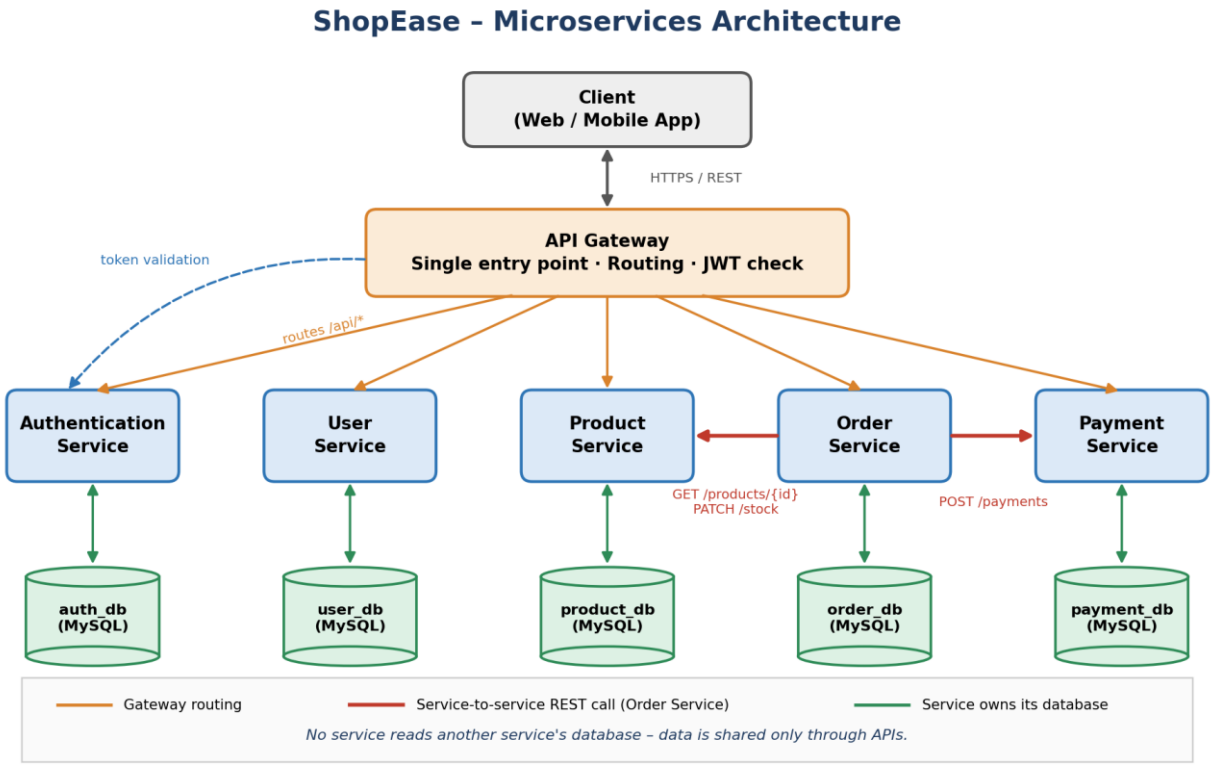


Figure 3: Complete ShopEase microservices architecture

7.1 Diagram Explanation

- The client (web or mobile app) sends all requests to the API Gateway only.
- The API Gateway validates the JWT through the Authentication Service and routes each request to the correct service.
- Each of the five services has its own MySQL database (auth_db, user_db, product_db, order_db, payment_db).
- The Order Service communicates with the Product Service (availability check and stock update) and the Payment Service (payment processing) over REST.
- No service accesses another service's database directly.
- In deployment, every service and database runs in its own Docker container, managed with Docker Compose.

8. Expected Project Flow

Step	Stage	Service involved
1	Customer registration and login	Authentication Service (and User Service for the profile)
2	JWT authentication	Authentication Service and API Gateway
3	Product browsing	Product Service
4	Order creation	Order Service
5	Product availability validation	Order Service calling Product Service
6	Payment processing	Order Service calling Payment Service
7	Order status update	Order Service
8	Order history and reporting	Order Service and Payment Service

9. Learning Summary

- Microservices split a large application into small, independent services, each with one responsibility.
- Each service owns its database and exposes data only through APIs.
- The API Gateway is the single entry point and handles routing and token verification.
- Services communicate with REST over HTTP and must handle failures using timeouts, retries and clear status codes.
- Day 1 deliverables are completed: comparison, service responsibilities, database ownership, REST notes, communication workflow and the architecture diagram. Implementation begins only after review.